Unit 3: The Problem-Solving Process & Solution Design

In this unit, we move from conceptual strategies to the formal methods of designing a solution. This is the "blueprint" stage of computing.

3.1 The 6-Step General Problem-Solving Process

Before writing a single line of code, a professional follow a systematic cycle to ensure the solution is robust and efficient:

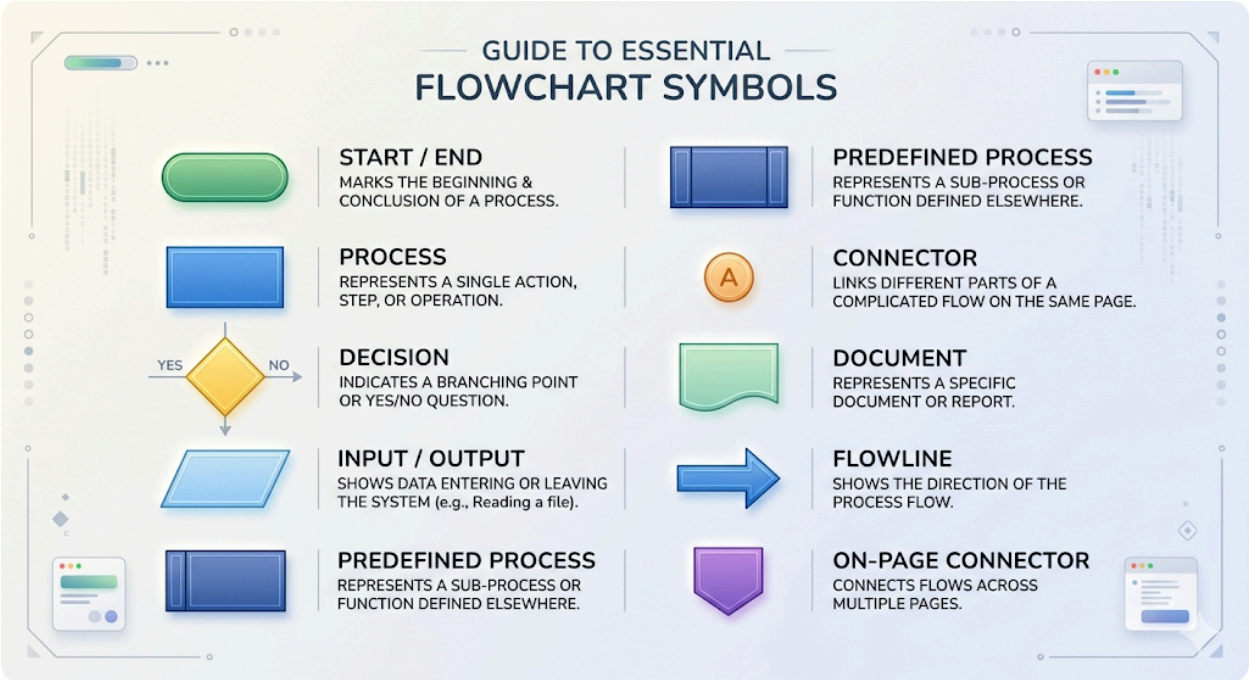
1. **Define the Problem:** Clearly state the goal and identify the inputs and expected outputs.
2. **Analyze the Problem:** Use the techniques from Unit 2 (like Root Cause Analysis or Abstraction) to understand constraints and requirements.
3. **Design the Solution:** Create a logical plan using tools like flowcharts or pseudocode.
4. **Implementation:** Translate the design into a programming language (C, Python, etc.).
5. **Evaluation & Testing:** Run the program with different data sets to ensure accuracy and handle errors.
6. **Refinement & Maintenance:** Improve the code for better performance and update it as requirements change.

3.2 Formal Design Tools: Flowcharts

A **flowchart** is a diagrammatic representation of an algorithm. It uses standard symbols to show the step-by-step flow of a process.

Standard Flowchart Symbols:

- **Oval (Terminal):** Represents the Start and End of a program.
- **Parallelogram (Input/Output):** Represents data entering or leaving the system.
- **Rectangle (Process):** Represents an action, calculation, or assignment (e.g., `x = a + b`).
- **Diamond (Decision):** Represents a condition or branch (e.g., "Is x > 10?"). It typically has two paths: Yes/No or True/False.
- **Arrows (Flowlines):** Indicate the direction of the process flow.



3.3 Formal Design Tools: Pseudocode

Pseudocode is an informal, high-level description of an algorithm. It uses the structural conventions of programming but is intended for human reading rather than machine execution.

- **Key Advantage:** It is "language-agnostic," meaning it works whether you eventually plan to code in C, Python, or Java.
- **Example (Finding the larger of two numbers):**
- Plaintext

```
START
  INPUT num1, num2
  IF num1 > num2 THEN
    PRINT num1
  ELSE
    PRINT num2
  ENDIF
END
```

3.4 Logic Mapping: Decision Tables and Trees

When a problem involves complex conditions and multiple possible actions, simple flowcharts can become cluttered.

- **Decision Table:** A tabular representation used to model logic based on different combinations of conditions and their resulting actions.
- **Decision Tree:** A graph that uses a branching method to illustrate every possible outcome of a decision.

3.5 Implementation, Evaluation, and Refinement

This is the bridge between design and the final product.

- **Implementation:** The actual coding phase.
- **Evaluation:** Does the solution meet the original requirements?
- **Refinement:** This involves **Optimization** (making it faster or using less memory) and **Debugging** (fixing logic errors identified during testing).